

Machine Learning Fundamentals - A Beginner's Guide for Data Analytics

Presented by -Nandigama Jeevan Sai

@Analytics career connect!

content

Introduction to Machine Learning

Supervised Learning Basics

Key Concepts and Terminology

Unsupervised learning basics

Evaluation metrics

Data preprocessing techniques

Introduction to model building Scikit-Learn

Model Tuning and Optimization

Introduction to Machine Learning

- ▶ **Machine learning** : machine learning is process of developing intelligent machine which can learn by themselves without explicitly of program .
- ▶ Machine learning term was coined or named by “Arthur Samuel” in 1959.
- ▶ **Types Machine learning**
 - **Supervised machine learning**

It is a mechanism in which machines are trained under supervision by providing labelled data .

- **Unsupervised machine learning**

It is mechanism in which model is trained without unlabeled data without any supervision .

- **Reinforcement machine learning**

It is mechanism in which model is trained with its own experience i.e trial and error basis . No data is required for this type machine learning .

machine learning importance in data analytics

- ▶ **Machine learning (ML) is pivotal in data analytics, transforming raw data into actionable insights.**
- 1. Automation of data processing
- 2. Pattern analytics
- 3. Predictive analytics
- 4. Real -time analytics
- 5. Scalability
- 6. Personalization
- 7. Improving accuracy
- 8. Insights from unstructured data
- 9. Reductio of human bias
- 10. Cross industry application

Supervised Learning Basics

Regression

- ▶ Regression is problem statement in which dependent column is continuous numerical data . i.e predicted output should a continuous numerical data .
- ▶ Algorithms .
 1. Linear regression
 2. Polynomial regression
- ▶ Metrics evaluation
 1. Mean absolute error
 2. Mean square error

Classification

- ▶ Classification is problem statement in which dependent column is categorical data. In this problem statement data is classified into different segments / groups / categories.
- ▶ Algorithms .
 1. Logistic regression(it for classification)
 2. Decision tree
- ▶ Metrics evaluation
 1. Accuarcy score
 2. F1 score

Applications

- Forecasting sales or revenue.
- Predicting environmental data (e.g., rainfall, temperature).
- Estimating the value of assets (e.g., cars, real estate).

Example :

Input: Number of rooms, location, and size of a house.

Output: Predicted price of the house (e.g., \$350,000).

Applications

- Spam detection in emails.
- Image recognition (e.g., cat vs. dog).
- Medical diagnosis (e.g., cancer/no cancer).

Example :

Input: Patient symptoms and test results

Output: Classifying whether the patient has a disease (yes/no).

- ▶ **Linear regression** : Estimating relationship between independent variables and dependent variables by drawing a straight line in dataset is called linear regression .
- ▶ **Best fit line** : A line which passes through maximum data points and at the same it is very closely from other data points.
- ▶ Best fit line → $y = mx + c$
- ▶ Y=dependent column
- ▶ X=independent column
- ▶ M=slope
- ▶ C= y-intercept
- ▶ **Types of linear regression**
 1. Simple linear
 2. Multi-linear regression



- **Simple linear regression** : Estimating relationship between single independent variables and single dependent variables by drawing a best fit line
- $Y=mx+c$
- **Multi-linear regression** : Estimating relationship between multiple independent variables and single dependent variables by drawing a best fit line
- $Y=mx+c$
- $Y=m_1x_1+m_2x_2+m_3x_3+.....m_nx_n+c$

Slope : Angle at which best fit line inclining . Or steepness of best fit line

Y-intercept : A point at which best fit line cross on y axis is called y-intercept .

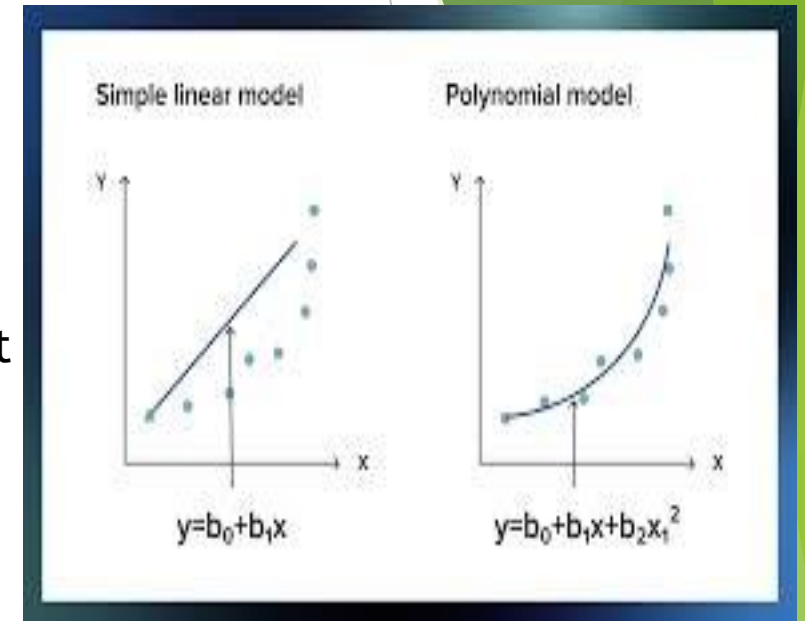
Slope can be calculated for dataset as

$$m = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})^2}$$

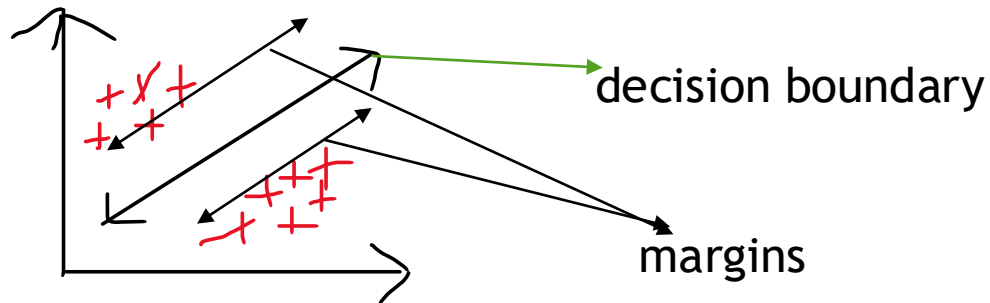
- ▶ **Polynomial regression** : It is process of finding the best fit line on non linear data with the help of polynomial feature by increasing degree
- ▶ $Y=c+m_1x_1+m_2x_1^{**2}+m_3x_1^{**3}.....+m_nx_1^{*****n}$

Support vector machine :

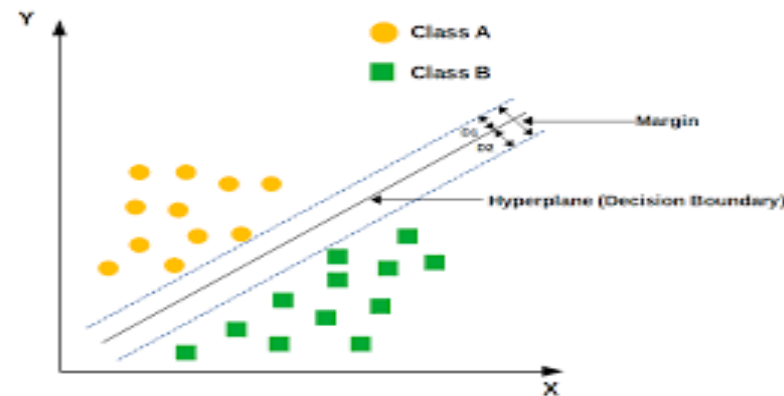
- ▶ It is supervised machine algorithm which is mainly developed for
- ▶ Classification problem statement by classifying data into segment
- ▶ Or different categories .
- ▶ SVM can be used for both classification and regression problem
- ▶ Statement
- ▶ It is very good for classification but not good in regression . SVM can classify the data by drawing a st line Which is called as **decision boundary**.



- Decision boundary : Decision boundary is st line which separates data into different clusters . These decision boundary should be placed at maximum distance from closest clusters of data points.
- In SVM decision boundary is called hyperplane .

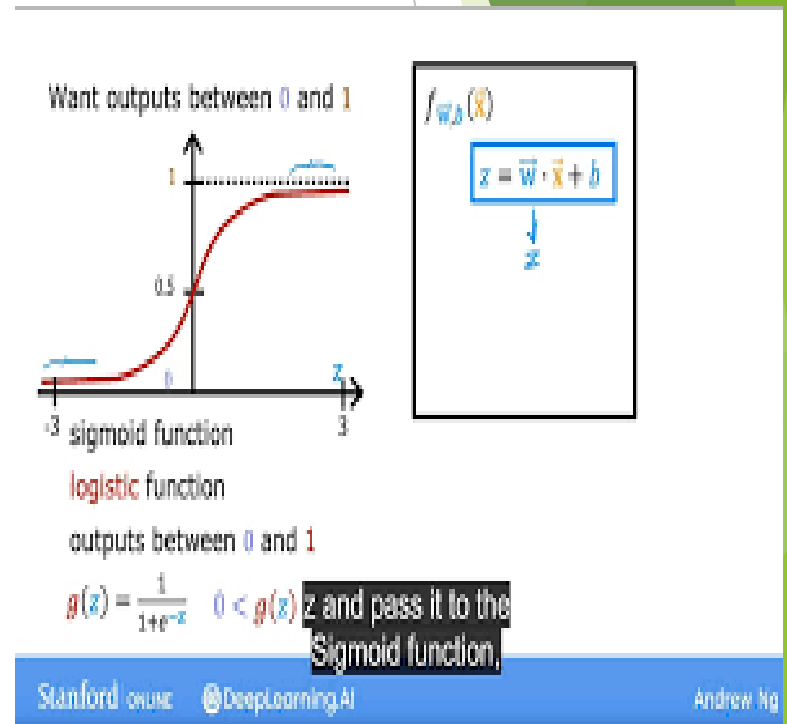


- Margins : margins are st line which are draw on either side of decision boundary which are passing through closest data points of each cluster is called margins . Distance between these margins are called margin distance .



- **Logistic regression** : Logistic Regression is a supervised learning algorithm used for classification tasks. Despite its name, it is not a regression algorithm but a classification model that predicts probabilities and assigns labels to data points based on those probabilities.
- It is designed to solve classification problem
- Solved by drawing a S-curved line is called sigmoid curve

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$



Key Concepts and Terminology

► Model training and testing :

Training : Training involves teaching the machine learning algorithm to recognize patterns in data. During training, the model : Learns from a labeled dataset (supervised learning) or uncovers patterns (unsupervised learning). Optimizes its parameters to minimize the error using a cost function.

➤ steps

1. Data Preparation:

Collect and preprocess the data (e.g., cleaning, normalization, feature engineering).
Split the dataset into training, validation, and testing sets (e.g., 70%-15%-15%).

2. Algorithm Selection: Choose an appropriate algorithm based on the problem (e.g., linear regression, decision trees, neural networks).

3. Training:

Feed the training data into the model.

Use optimization techniques (e.g., gradient descent) to adjust the model parameters.

4. Hyperparameter Tuning:

Adjust parameters not learned during training (e.g., learning rate, regularization strength) to improve performance.

► **Model Testing :**

Testing evaluates the model's performance on unseen data to ensure it generalizes well and performs reliably in real-world scenarios.

► **Steps in Model Testing:**

1. **Testing Dataset:**

Use a separate dataset that the model has not seen during training or validation.

2. **Prediction:**

Run the model on the test dataset to generate predictions.

3. **Evaluation:**

Compare predictions against actual outcomes using performance metrics

Importance of Splitting Data

Training Set: Used to train the model.

Validation Set: Used to tune hyperparameters and avoid overfitting.

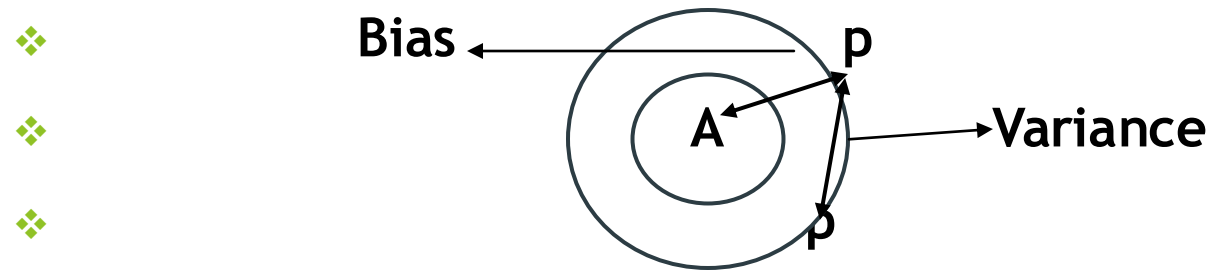
Test Set: Used to assess the final performance of the model.

Splitting ensures the model is tested on data it has never seen, preventing over-optimistic performance evaluations.

► Bias and variance :

❖ Bias : Difference between actual value and predicted value is called bias.

❖ Variance : Distance between two predicted values is called variance.



➤ Data with high bias and data with low variance → Under fitting

➤ Data with low bias and data with high variance → Over fitting

➤ Data with high bias and data with high variance → worst

➤ Data with low bias and data with low variance → perfect data model

► Over fitting

It is stage where model learns 100% from training data but unable to perform prediction is called overfitting.

Data with high variance and low bias

Check for test score : **low(30)**

Check for train score : **high (80-90)**

Reason for overfitting :

1. Less data volume
2. Data with large feature
3. Data with high variance

Solution for overfitting : **Regularzitation**

► Under fitting :

It is stage where model is not able to learn from training data nor can perform prediction is called underfitting

Check for test score : low (33)

Check for train score : low around test data score (35)

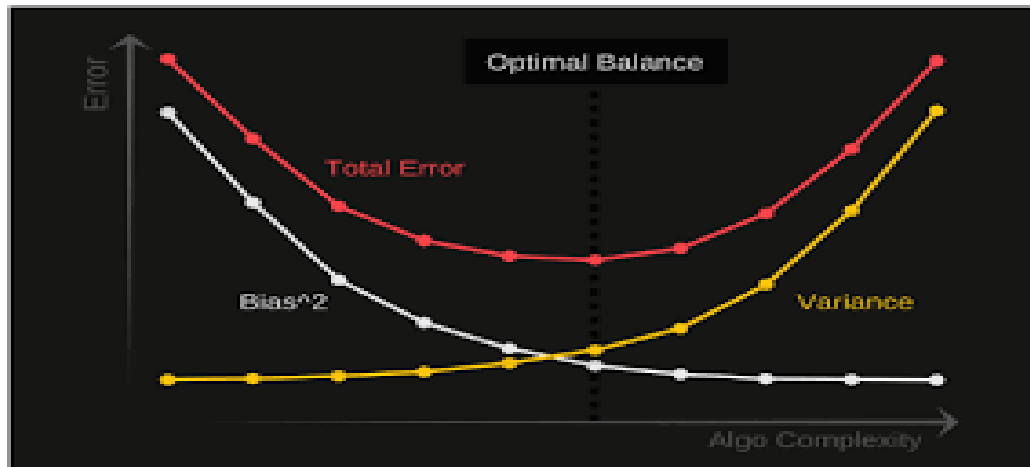
► Reason for underfitting :

1. Very less volume of data
2. Less feature
3. Data with high bias

Solution for underfitting : **polynomial regression**

Bais - variance trade off :

Bais is equal to variance is called bias- variance trade off



Unsupervised learning basics

- Unsupervised machine learning

It is mechanism in which model is trained without unlabeled data without any supervision .

- ▶ Model classify the data based on common features
- ▶ These feature extracted by model itself.

Problem statement for unsupervised learning

1. Clustering problem statement
2. Dimensionality reduction
3. Associate mining

Clustering : Clustering is an unsupervised machine learning technique used to group data points into clusters, such that points within the same cluster are more similar to each other than to those in other clusters. It is widely used in various fields like data mining, pattern recognition, image analysis, and bioinformatics.

► **Types of clustering :**

1. **K-means clustering**
2. **Hierarchical clustering**
3. **Density based clustering**
4. **Mean shift clustering**
5. **Spectral clustering**

► **K-means clustering :** K-Means Clustering is one of the most widely used clustering algorithms. It is a centroid-based algorithm that aims to partition a dataset into k clusters by minimizing the variance within each cluster.

► **Elbow method in k-means clustering :**

► **The Elbow Method** is a popular technique for determining the optimal number of clusters (k) in K-Means Clustering. It evaluates the performance of the clustering by measuring the Within-Cluster Sum of Squares (WCSS) for different values of k , then selecting the k where the WCSS starts to diminish at a slower rate, resembling an "elbow" in the curve.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs

# Generate synthetic data
X, _ = make_blobs(n_samples=300, centers=4, cluster_std=0.6, random_state=0)

# List to store WCSS values
wcss = []

# Compute K-Means for k = 1 to 10
for k in range(1, 11):
    kmeans = KMeans(n_clusters=k, random_state=0)
    kmeans.fit(X)
    wcss.append(kmeans.inertia_) # WCSS is stored in kmeans.inertia_

# Plot the Elbow Method
plt.figure(figsize=(8, 5))
plt.plot(range(1, 11), wcss, marker='o', linestyle='--')
plt.title('Elbow Method')
plt.xlabel('Number of Clusters (k)')
plt.ylabel('WCSS')
plt.xticks(range(1, 11))
plt.grid()
plt.show()
```



► Dimensionality reduction :

Dimensionality reduction is a method for representing a given dataset using a lower number of features (i.e. dimensions) while still capturing the original data's meaningful properties.¹ This amounts to removing irrelevant or redundant features, or simply noisy data, to create a model with a lower number of variables.

► Why use dimensionality reduction?

In machine learning , dimensions (or features) are the predictor variables that determine a model's output. They may also be called input variables. High-dimensional data denotes any dataset with a large number of predictor variables. Such datasets can frequently appear in biostatistics, as well as social science observational studies, where the number of data points (i.e. observations) outweighs the number of predictor variables.

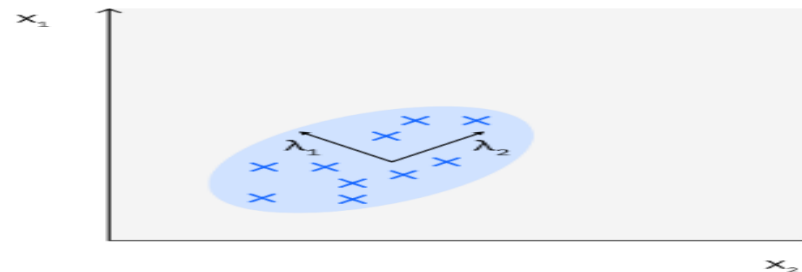
High-dimensional datasets pose a number of practical concerns for machine learning algorithms, such as increased computation time, storage space for big data, etc. But the biggest concern is perhaps decreased accuracy in predictive models. Statistical and machine learning models trained on high-dimensional datasets often generalize poorly.

► Principal component analysis

Principal component analysis (PCA) is perhaps the most common dimensionality reduction method. It is a form of feature extraction, which means it combines and transforms the dataset's original features to produce new features, called principal components. Essentially, PCA selects a subset of variables from a model that together comprise the majority or all of the variance present in the original set of variables. PCA then projects data onto a new space defined by this subset of variables.

For example, imagine we have a dataset about snakes with five variables: body length (X_1), body diameter at widest point (X_2), fang length (X_3), weight (X_4), and age (X_5). Of course, some of these five features may be correlated, such as body length, diameter, and weight. This redundancy in features can lead to sparse data and overfitting, decreasing the variance (or generalizability) of a model generated from such data. PCA calculates a new variable (PC1) from this data that conflates two or more variables and maximizes data variance. By combining potentially redundant variables, PCA also creates a model with less variables than the initial model. Thus, since our dataset started with five variables (i.e. five-dimensional), the reduced model can have anywhere from one to four variable (i.e. one- to four-dimensional). The data is then mapped onto this new model.

PCA:



Evaluation metrics

Regression Metrics

- ▶ Regression metrics evaluate how well a regression model predicts continuous outcomes. Common metrics include:
- ▶ Mean Absolute Error (MAE):
 - Definition: The average of the absolute differences between predicted and actual values.

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

- ▶ Mean Squared Error (MSE):
 - Definition: The average of the squared differences between predicted and actual values.

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

► . R-Squared (R^2):

- **Definition:** The proportion of variance in the dependent variable that is predictable from the independent variables.

$$R^2 = 1 - \frac{SS_{\text{residual}}}{SS_{\text{total}}}$$

► **Classification Metrics**

Classification metrics assess a model's ability to classify discrete labels correctly. They are often derived from a **confusion matrix**

► **confusion Matrix:**

A matrix that summarizes the performance of a classification model:

Actual/Predicted	Positive	Negative
Positive	True Positive (TP)	False Negative (FN)
Negative	False Positive (FP)	True Negative (TN)

► **Accuracy:**

- **Definition:** The ratio of correct predictions to total predictions.

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{Total Observations}}$$

- **Interpretation:** Useful for balanced datasets, but misleading for imbalanced datasets.
- **Use Case:** Classifying spam emails when spam and non-spam emails are balanced

► **Precision:**

Definition: The proportion of positive predictions that are actually correct.

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

Interpretation: High precision means fewer false positives.

Use Case: Detecting fraudulent transactions where false positives lead to unnecessary investigations

► **Recall (Sensitivity):**

- **Definition:** The proportion of actual positives that are correctly predicted.

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

Interpretation: High recall means fewer false negatives.

Use Case: Diagnosing diseases where missing true cases is costly

► **F1-Score:**

- **Definition:** The harmonic mean of precision and recall, balancing the trade-off between them.

$$\text{F1-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Interpretation: A good metric when the dataset is imbalanced.

Use Case: Predicting loan defaults where both false positives (flagging non-defaulters) and false negatives (missing actual defaulters) are important.

Data preprocessing techniques

1. Data cleaning : identifies and removes errors ,outliers, and inconsistence data .

➤ Handling missing data :

To identify missing values function like `.isnull()` or `.info()`

To remove missing values in row and columns function like `df.dropna()`

To replace missing values ,for numerical data we use `mean(),mode(),median()` with `fillna` function
for time- series data we use `bfill` or `ffill` method with `fillna` function .

`df["column"].fillna(df["column"].mean()/mode()/median() or method="ffill/bfill)`

➤ Removing duplicates :

•Check for and remove duplicate rows to avoid bias.

•Example: `df.drop_duplicates()`

➤ Handling outliers :

By using IQR method and mean-standard deviation method

For IQR

`q1=df["column"].quantile(0.25)`

`q3=df["column"].quantile(0.75)`

`IQR=q3-q1`

`upper limit = q3+(1.5*IQR), lower limit=q1-(1.5*IQR)`

`df["column"]=df["column"].map(lambda x: upper limit if x>upper limit else lower limit if x<lower limit else x)`

For mean-standard deviation

```
Mean = df["column"].mean()
```

```
Std=df["column"].std()
```

```
Up=mean+3*std
```

```
Lp=mean-3*std()
```

```
df["column"]=df["column"].map(lambda x: Up if x>Up else Lp if x<Lp else x)
```

2. **Feature engineering** : Feature Engineering is the process of transforming raw data into meaningful features that improve the performance of machine learning models. It plays a crucial role in determining the success of a machine learning project .

Key techniques for feature engineering

1. **Handling categorical variables**

- **Encoding** : it is process of converting categorical data into numerical form
categorical data → non numerical form

- One hot encoder : Converting data into binary columns

example : `pd.get_dummies(df['category'])` or

`from sklearn.preprocessing import OneHotEncoder`

- Label encoder : Assign unique integer to each category

Example : `from sklearn.preprocessing import LabelEncoder`

3. Feature scaling :

- ▶ Standardization : Scale features to have a mean of 0 and a standard deviation of 1

Example: Using StandardScaler from sklearn

- ▶ Normalization :

Scale features to a range (e.g., 0 to 1).

Example: Using MinMaxScaler from sklearn

► Creating New Features

1. Domain-Specific Features

Example: From a timestamp, create features like hour, day, or month.

2. Mathematical Transformations

Apply log, square, or square-root transformations to normalize skewed data.

Example: `df['log_feature'] = np.log1p(df['feature'])`

3. Aggregations

Group by categories and compute statistics (mean, median, sum, etc.)

4. Interaction Features

Combine features using arithmetic operations

Example: `df['interaction'] = df['feature1'] * df['feature2']`

4. Handling Date/Time Data:

Extract features like year, month, day, hour, day of the week, etc., from a datetime column.

```
import pandas as pd
import numpy as np

# Sample dataset
data = {
    'age': [25, 30, np.nan, 40, 35],
    'salary': [50000, 60000, 75000, None, 52000],
    'category': ['A', 'B', 'A', 'C', 'B'],
    'date': ['2023-01-01', '2023-01-02', '2023-01-03', '2023-01-04', '2023-01-05']
}
df = pd.DataFrame(data)

# Fill missing values
df['age'].fillna(df['age'].median(), inplace=True)
df['salary'].fillna(df['salary'].mean(), inplace=True)

# One-hot encode categorical variable
df = pd.get_dummies(df, columns=['category'], drop_first=True)

# Create new features from date
df['date'] = pd.to_datetime(df['date'])
df['day_of_week'] = df['date'].dt.dayofweek

# Scale numerical features
from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler()
df[['age', 'salary']] = scaler.fit_transform(df[['age', 'salary']])

print(df)
```

Introduction of model building with scikit-learn

1. Import libraries

import necessary libraries for data handling, model building, and evaluation

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report
```

2. Load and explore data

```
# Load dataset
df = pd.read_csv('data.csv')

# Inspect data
print(df.head())
print(df.info())
print(df.describe())
```

3. Preprocessing data

Handle missing values, encode categorical variables, and scale numerical features.

```
# Fill missing values
df.fillna(df.median(), inplace=True)

# Encode categorical variables
df = pd.get_dummies(df, drop_first=True)

# Separate features and target
X = df.drop('target', axis=1) # Features
y = df['target']             # Target variable
```

4. Split Data into Training and Testing Sets

Why is Train-Test Split Important? Splitting data ensures the model is evaluated on unseen data, simulating its performance in real-world scenarios.

```
from sklearn.model_selection import train_test_split

# Split the data (80% training, 20% testing)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

print(f"Training set size: {X_train.shape[0]}")
print(f"Testing set size: {X_test.shape[0]}")
```

► Build and Train the Model

Select a machine learning algorithm and fit the model to the training data.

```
# Initialize the model
model = RandomForestClassifier(random_state=42)

# Train the model
model.fit(X_train, y_train)
```

6. Make Predictions

Use the trained model to make predictions on the test set.

```
# Predict on the test set
y_pred = model.predict(X_test)
```

7. Evaluate the Model

Use metrics like accuracy, precision, recall, and F1-score to evaluate model performance.

```
# Accuracy
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy:.2f}")

# Classification Report
print("Classification Report:")
print(classification_report(y_test, y_pred))
```

8. Cross-Validation :

It is technique through which model is trained maximum from data by splitting in all possible ways.

```
from sklearn.model_selection import cross_val_score

# Perform 5-fold cross-validation
cv_scores = cross_val_score(model, X, y, cv=5)
print(f"Cross-validation scores: {cv_scores}")
print(f"Mean CV score: {np.mean(cv_scores):.2f}")
```

Model Tuning and Optimization

○ Hyperparameter Tuning: Hyperparameters are settings that are not learned from the data but are defined before training a model. Examples include the number of trees in a Random Forest or the learning rate in Gradient Boosting.

Techniques for Hyperparameter Tuning :

Grid Search

Random Search

► 2. Ensemble Techniques

- Ensemble methods combine multiple models to improve overall accuracy and robustness.

Bagging (Bootstrap Aggregating) :

Combines predictions from multiple models trained on different random subsets of data.

Example: Random Forest Random Forest is a bagging method that builds multiple decision trees and averages their predictions.

► **Boosting**

- Sequentially trains weak models (e.g., shallow decision trees) to correct the errors of previous models.

**THANK
YOU!**